

Projet d'électronique S6 :

Système de buzzers interactifs

Sommaire

1. Introduction	2
1.1 Cahier des charges	2
1.2 Schéma architectural du système	2
2. Software	3
2.1 Configuration matérielle sous STM32CubeIDE	3
2.2 Protocole de communication entre les cartes (CAN)	4
2.3 Fonctionnalité supplémentaire: interface PC et simulation de clavier (USB HID)	5
2.4 Algorithme principal et logique de jeu	6
2.5 Test du code à partir d'une liaison CAN fournie	7
3. Hardware	8
3.1 Bus de communication et synchronisation	8
3.2 Interface USB-C et alimentation	10
3.3 Interfaces utilisateur	12
4. Mécanique	14
4.1 Architecture globale du boîtier	14
4.2 Choix technologique : Le mécanisme de déclenchement	14
4.3 Intégration Hardware et Ergonomie	15
4.4 Assemblage final et perspectives de fabrication	16
5. Conclusion	16
Responsabilité environnementale (Up-Cycling)	17
Perspectives d'évolutions futures	17
ANNEXE	18

1. Introduction

Ce projet consiste à construire un système de buzzers interactifs.

Cette construction se divise en 3 parties. L'une portera sur le software par l'intermédiaire de STM32CubeIDE, qui se suit sur une partie hardware sous KiCad et enfin une partie mécanique en découvrant le logiciel OnShape.

1.1 Cahier des charges

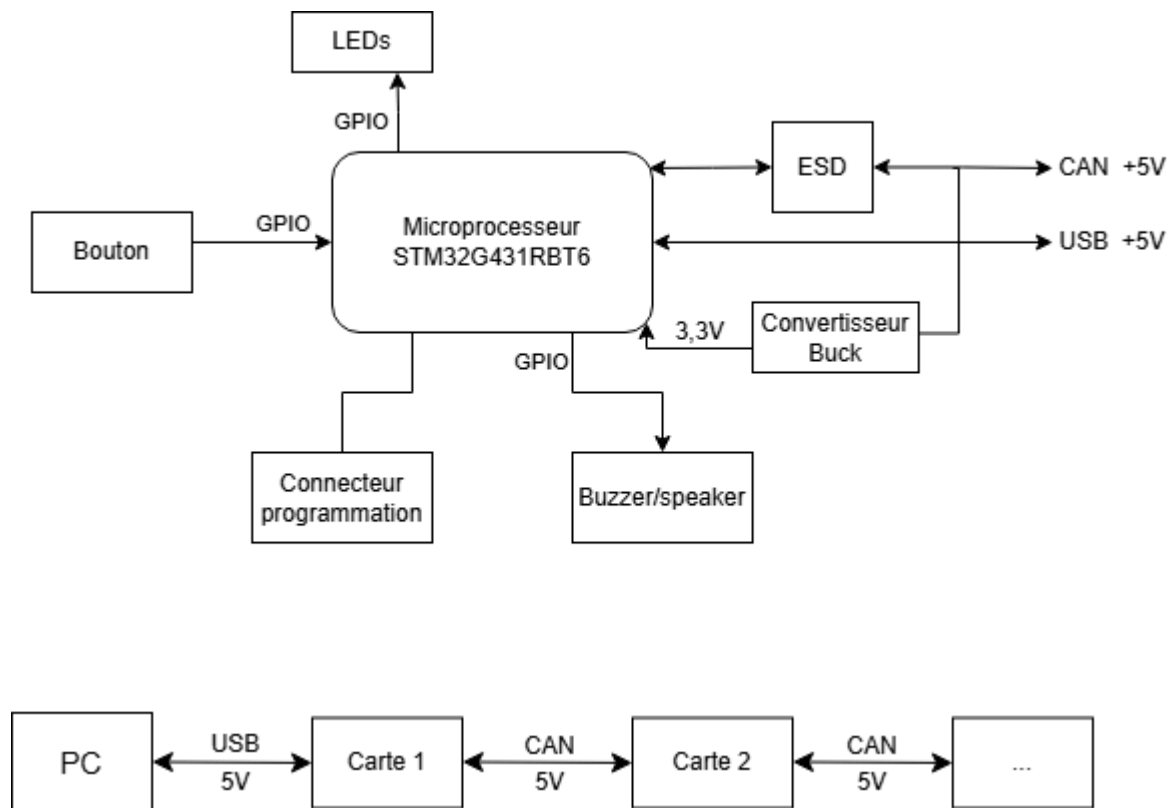
L'objectif est de répondre aux besoins suivants:

- Développer un modèle de buzzers interconnectés
- Il faut que 8 buzzers puissent fonctionner ensemble
- Affichage lumineux sur le premier Buzzer actionné
- Liaison filaire (Bus)
- Objectif secondaire : Contrôle avec PC (pour arrêter la musique par exemple)

1.2 Schéma architectural du système

Afin de répondre à ce cahier des charges, nous avons défini l'architecture globale du système. Le schéma ci-dessous illustre l'organisation des différents composants matériels autour du microcontrôleur central, ainsi que la façon dont les cartes communiquent entre elles et sont alimentées.

Figure 1: schéma d'architecture du système Buzzer



2. Software

2.1 Configuration matérielle sous STM32CubeIDE

Pour initier ce projet, nous avons utilisé STM32CubeIDE afin de générer le code d'initialisation de notre microcontrôleur et de configurer les différents périphériques internes du STM32G431RBT6. Comme l'illustre la figure de configuration du brochage (Figure 2), nous avons assigné des étiquettes explicites aux ports GPIO nécessaires : la lecture du bouton poussoir (PA10) et le pilotage des LEDs et du speaker (PA6, PB11, PB10).

Le réseau de communication entre les cartes est géré en activant le périphérique FDCAN1. L'horloge interne RTC a également été initialisée pour servir de base temporelle à la génération de nombres aléatoires. Enfin, notre interface PC repose sur l'activation de USB_Device, paramétré en Human Interface Device (HID). Pour que ce protocole USB

fonctionne correctement, nous avons veillé à configurer précisément l'arbre d'horloge afin de fournir la fréquence stricte de 48 MHz exigée par ce périphérique (Figure 3).

Figure 2: Configuration du STM32G431RBT6

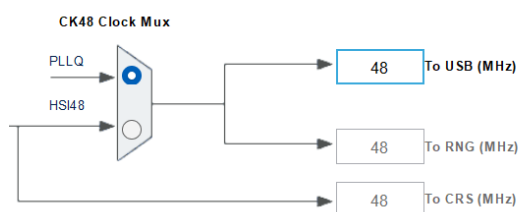
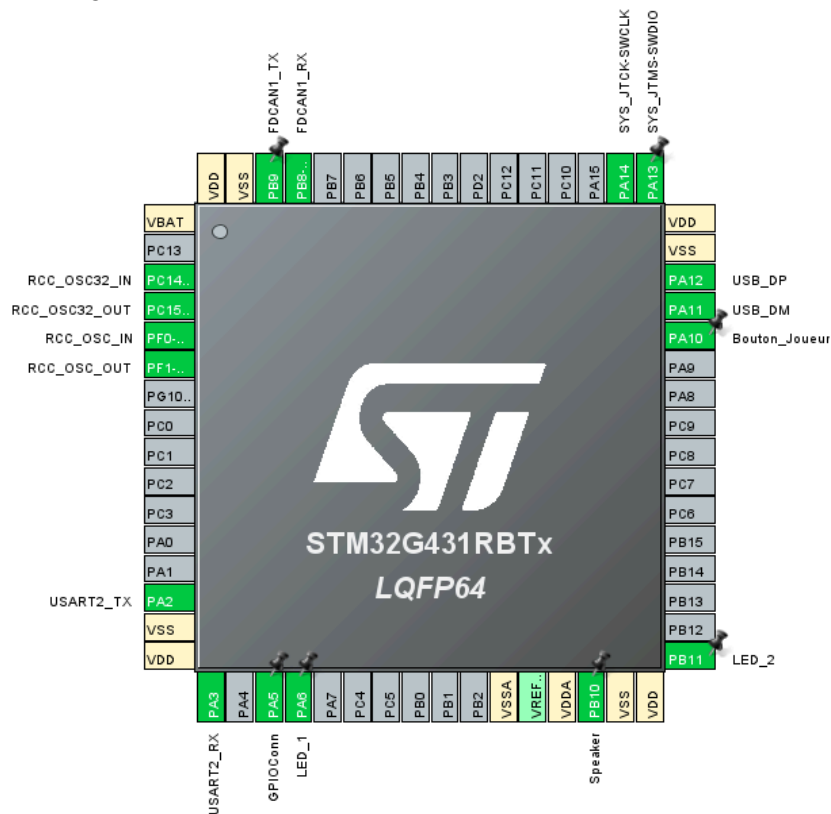


Figure 3: Zoom sur la configuration d'horloge - fréquence du périphérique USB

2.2 Protocole de communication entre les cartes (CAN)

Dans le cadre d'un système de blindtest, les différents buzzers doivent communiquer entre eux de manière fiable et réactive, sans nécessiter de serveur central. Le protocole CAN (ici FDCAN pour STM32) est idéal pour cette architecture. Le défi principal résidait dans la gestion des collisions : si deux joueurs appuient exactement en même temps, il faut éviter que les cartes envoient le même identifiant de message sur le bus, ce qui provoquerait une erreur.

Pour pallier ce problème, nous avons implémenté la fonction `generateRandomStdId()`. Celle-ci exploite l'horloge temps réel RTC de la carte pour créer une graine temporelle, générant ainsi un identifiant de message unique sur 11 bits pour chaque cycle de jeu.

Du côté de la réception, le STM32 fonctionne de manière asynchrone grâce aux interruptions. Lorsqu'un message FDCAN est détecté sur le bus (indiquant qu'un adversaire a été le plus rapide), la fonction `HAL_FDCAN_RxFifo0Callback` est déclenchée en arrière-plan. Cette routine d'interruption passe immédiatement la variable globale `system_blocked` à 1 et date cet événement via `HAL_GetTick()`. Le signal FDCAN généré par notre code est ensuite transmis au composant TJA1050 pour être converti en signal électrique sur le câble, ce qui sera détaillé dans la partie hardware.

```
//Un adversaire a buzzé: callback lors de la réception d'un message FDCAN
void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
{
    if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_NEW_MESSAGE) != RESET)
    {
        if (HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK)
        {
            //Quelqu'un a buzzé
            system_blocked = 1;
            last_block_time = HAL_GetTick();

            //On met en pause l'ordinateur
            send_spacebar();

            HAL_UART_Transmit(&huart2, (unsigned char*)UART_buf, strlen(UART_buf), 100);
        }
    }
}
```

2.3 Fonctionnalité supplémentaire: interface PC et simulation de clavier (USB HID)

L'une des fonctionnalités innovantes de notre système est sa capacité à contrôler directement l'ordinateur de l'animateur pour stopper la musique ou la vidéo lors d'un appui.

La carte est reliée au PC via un connecteur USB-C. Grâce à la configuration HID, le STM32 se fait passer pour un vrai clavier physique aux yeux de l'ordinateur. La fonction

send_spacebar() prépare un tableau de données au format standard des claviers USB. Elle y insère le code hexadécimal 0x2C (correspondant à la touche "espace"), l'envoie via le bus USB puis transmet une seconde trame vide pour simuler le relâchement de la touche. Ainsi, le logiciel de lecture multimédia est mis en pause instantanément dès le premier appui valide sur le réseau.

```
//Fonction pour mettre la vidéo/son en pause via USB (clavier HID)

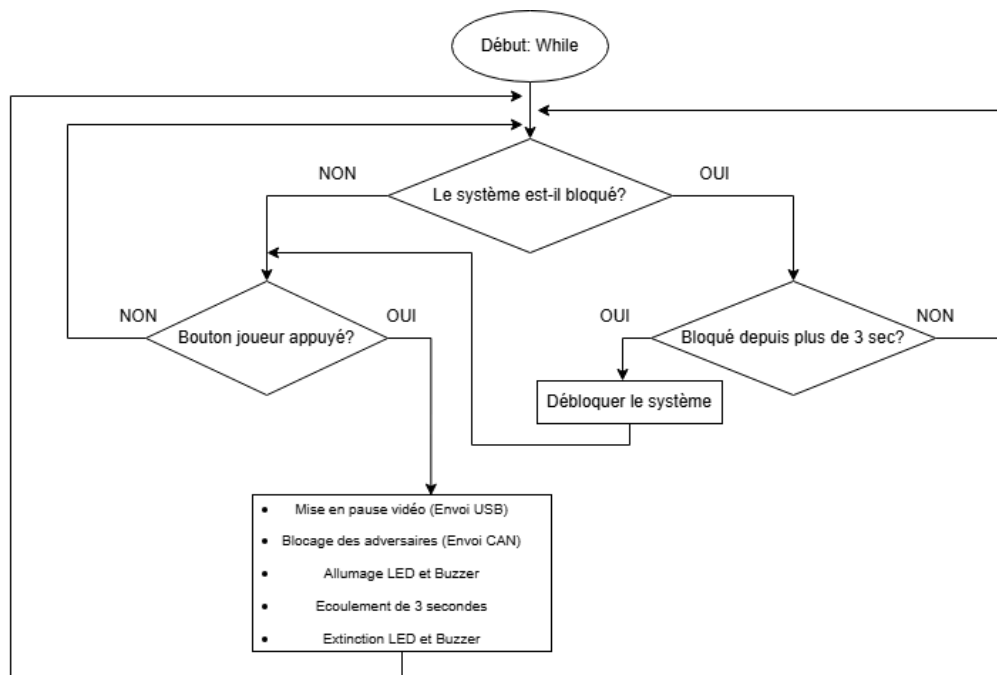
void send_spacebar(void) {
    uint8_t hid_report[4] = {0};

    //Appui sur la touche Espace (code HID:0x2C)
    hid_report[2] = 0x2C;
    USBD_HID_SendReport(&hUsbDeviceFS, hid_report, sizeof(hid_report));

    HAL_Delay(30); //Petit délai pour simuler un vrai doigt

    //Relâcher la touche
    hid_report[2] = 0x00;
    USBD_HID_SendReport(&hUsbDeviceFS, hid_report, sizeof(hid_report));
}
```

2.4 Algorithme principal et logique de jeu



Algorithme de la boucle d'exécution principale

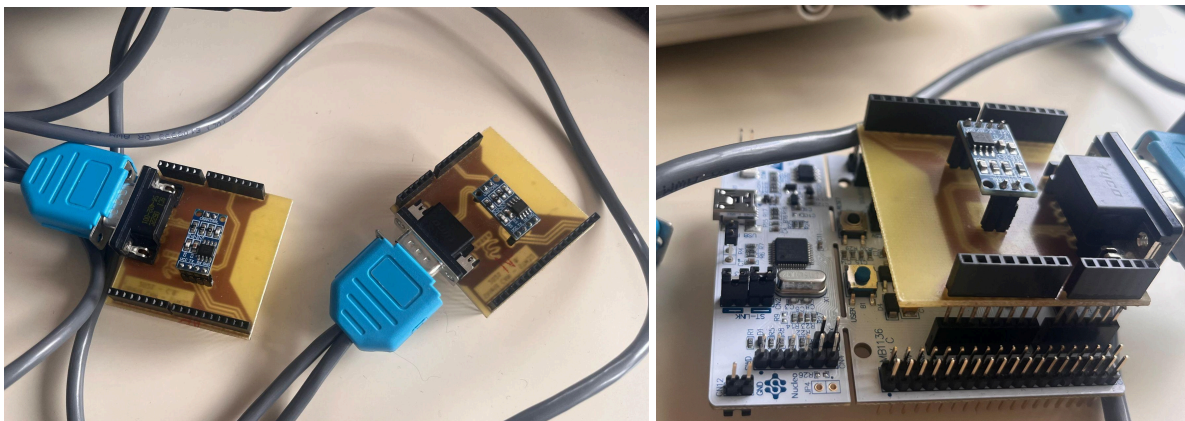
L'architecture logicielle de notre application repose sur une boucle infinie `while(1)`, dont le comportement est modélisé par l'algorithme ci-dessus (et dont le code complet figure en annexe).

À chaque itération, le STM32 vérifie l'état de la variable de verrouillage (`system_blocked`, cf Annexe). Si le système a été verrouillé suite à la victoire d'un adversaire, le microcontrôleur patiente jusqu'à ce que le délai de blocage de 3 secondes soit écoulé pour se réinitialiser.

Dans le cas où le système n'est pas bloqué, la carte scrute l'état du bouton poussoir. Dès qu'un appui est détecté (le joueur a buzzé), une séquence d'actions s'enclenche : la carte verrouille son propre système pour ignorer les rebonds, appelle la fonction USB pour mettre en pause le média de l'animateur, et diffuse instantanément son identifiant sur le bus CAN pour bloquer toutes les cartes adverses. Le speaker et les LEDs sont par ailleurs activés pour signaler la victoire. Après 3 secondes d'attente, le système éteint les périphériques de signalisation et prépare un nouvel identifiant aléatoire, marquant la disponibilité pour un nouveau tour de jeu.

2.5 Test du code à partir d'une liaison CAN fournie

Nous nous sommes finalement proposés de tester le bon fonctionnement du code grâce à une liaison CAN fournie ainsi que deux adaptateurs, fournissant des ports CAN à deux cartes Nucleo standards dont nous disposons. En adaptant notre code au processeur STM32L476 que contiennent ces dernières, nous avons ainsi vérifié, à travers l'allumage de ports connus (en l'occurrence la LED verte PA5 et le bouton poussoir PC13), que l'appui du bouton sur l'une des cartes (pour allumer la LED) bloque bien l'appui et allumage de l'autre pour une durée de 3 secondes.



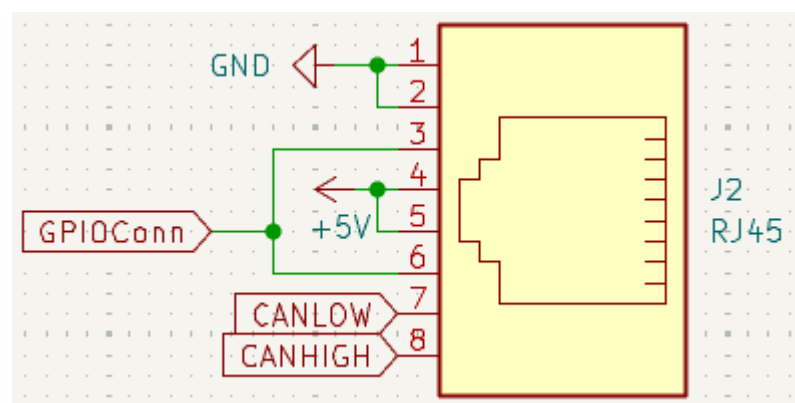
3. Hardware

Maintenant que la partie software est mise en place, il est nécessaire de programmer le microcontrôleur par un connecteur 14 pins, permettant de connecter chaque buzzer par un bus CAN et de relier le système interactif à un ordinateur en plus des fonctionnalités supplémentaires ajoutés tels que des signaux lumineux et sonores.

3.1 Bus de communication et synchronisation

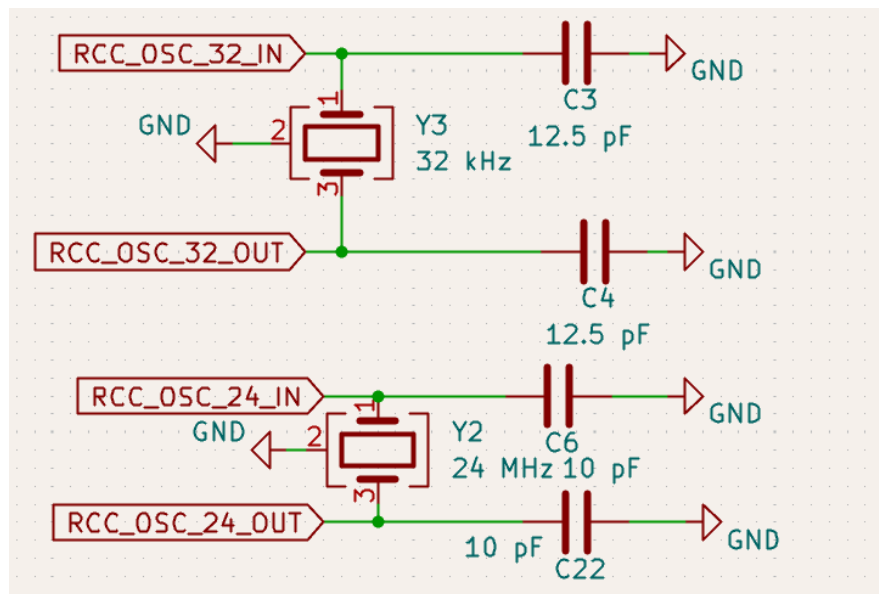
Pour assurer la communication entre les différents buzzers, nous avons implémenté un bus CAN (Controller Area Network). La décision s'est portée sur 2 connecteurs RJ45 SS-6388-NF servant de récepteur et de renvoi de données, ainsi que des câbles Ethernet de catégorie 5.

L'utilisation des paires torsadées permet de transporter le signal différentiel Can High / Can Low en maintenant une robustesse aux bruits et d'alimenter (5V / GND) chacun des buzzers.

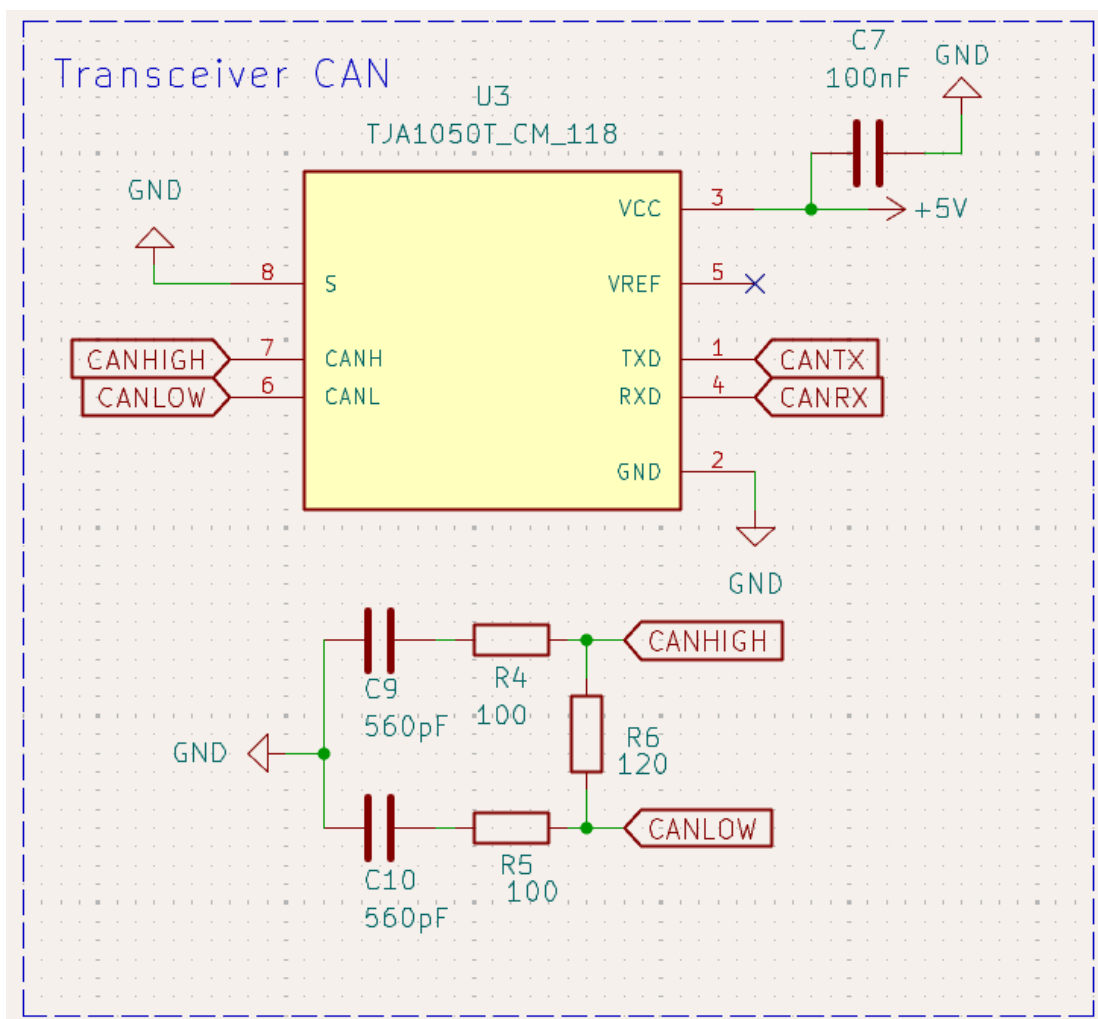


Ces connecteurs RJ45 sont placés au bord du PCB afin de gagner de la place pour les composants restants.

De plus, il est important de mettre en place 2 quartz, relié au microcontrôleur dont l'un de 24MHz pour la synchronisation des données rapides via le bus CAN et l'USB-C que nous allons aborder et un deuxième de 32 KHz permettant au microcontrôleur de garder l'heure exacte.



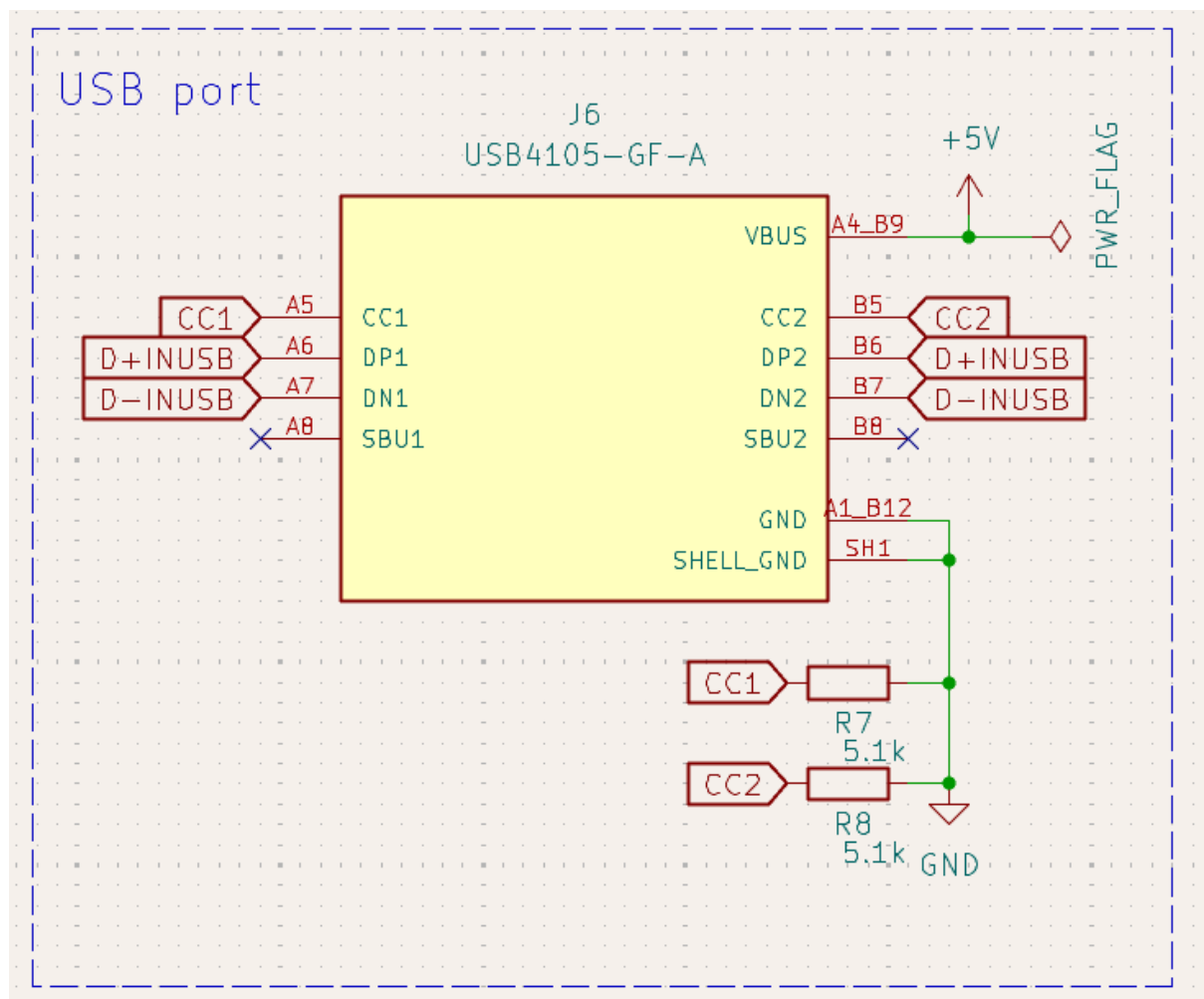
Enfin, cette transcription des signaux logiques du microcontrôleur en signal différentiel est assurée par le transceiver CAN TJA1050T_CM_118.



La résistance R6 du filtre ajouté supprime les phénomènes de réflexion des signaux électriques. Elle est choisie selon l'impédance du câble, en l'occurrence 120 Ω dans notre cas.

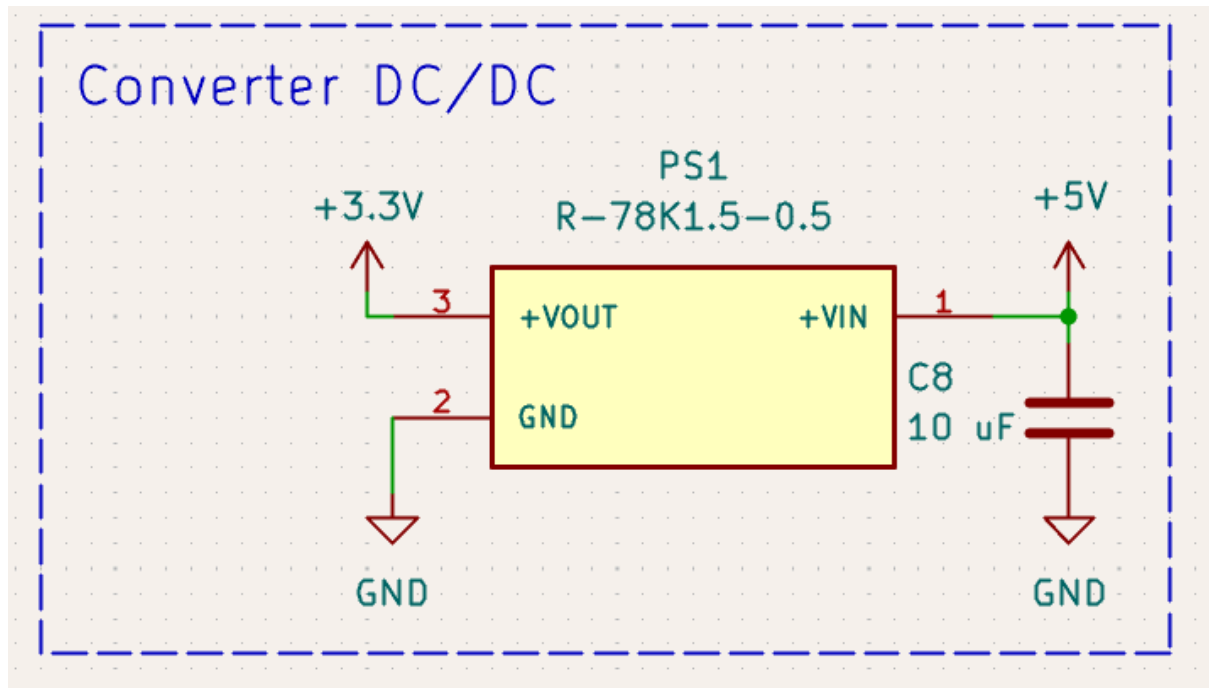
3.2 Interface USB-C et alimentation

La connexion USB-C du premier buzzer avec l'ordinateur fournit une tension standard de 5V. Nous avons choisi le USB4105-GF-A.



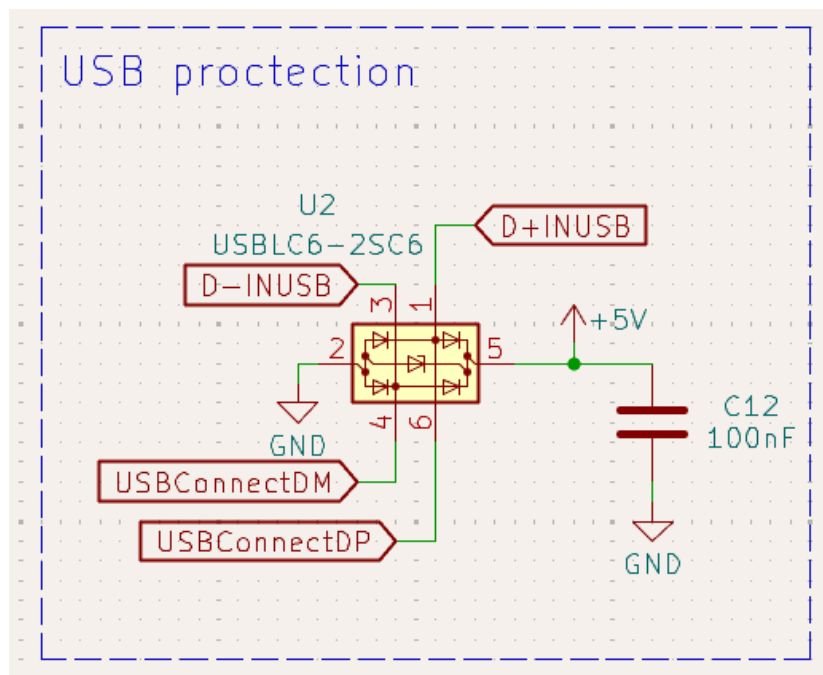
Identiquement que pour les ports RJ45, le connecteur USB-C est placé au bord du circuit imprimé.

Cette tension est ensuite convertie en 3.3 V par un convertisseur DC/DC de référence R-78K1.5-0.5 pour alimenter le microcontrôleur et les composants passifs, notamment les condensateurs de filtrage du microcontrôleur.



Il est routé au plus proche du port USB-C pour générer une alimentation filtrée et stable.

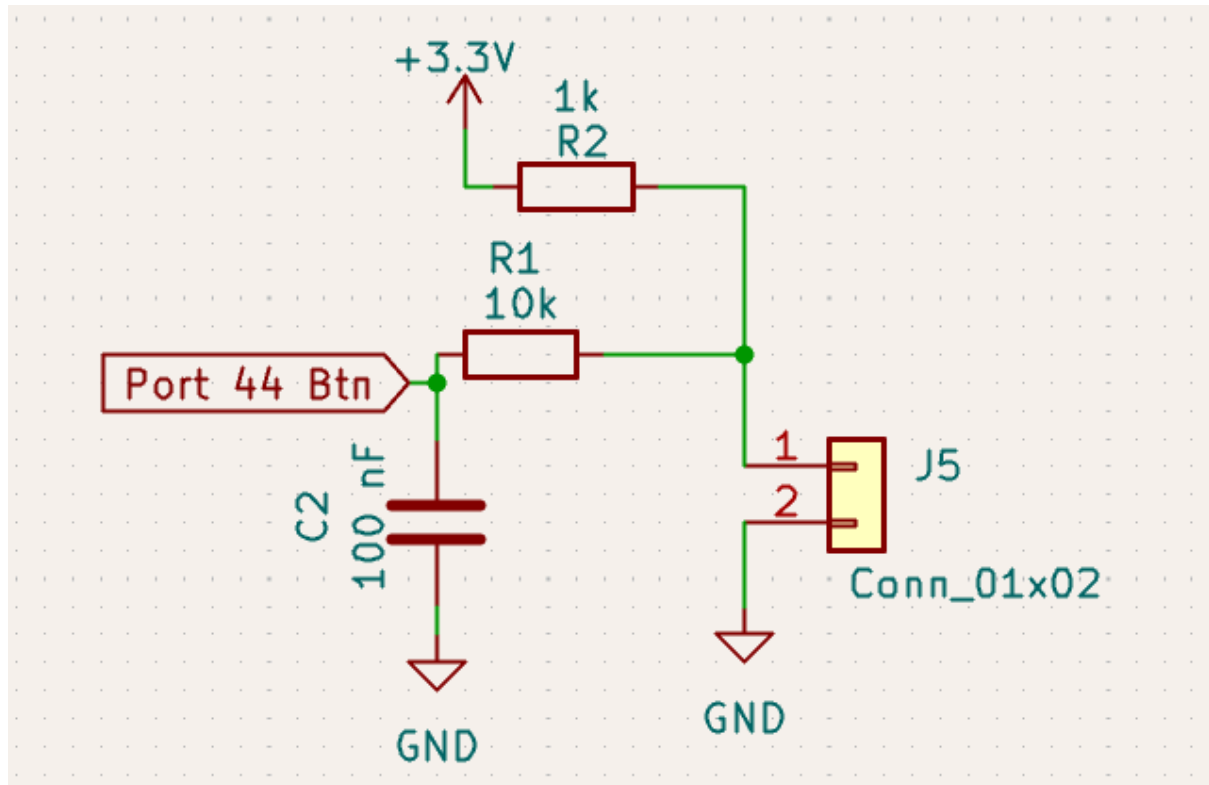
En outre, il est important d'ajouter une protection USB sur ce connecteur USB-C car les débranchements du câble USB provoquent des décharges électrostatiques (ESD), endommageant les ports du microcontrôleur.



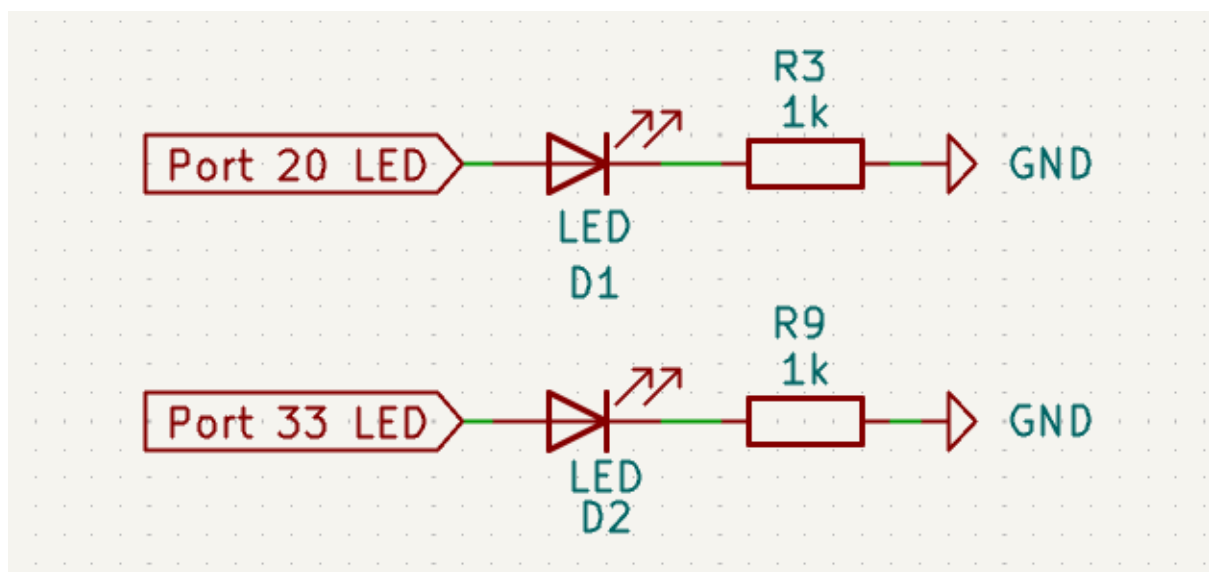
De même, il est routé au plus près du port USB-C afin de limiter la zone de ESD.

3.3 Interfaces utilisateur

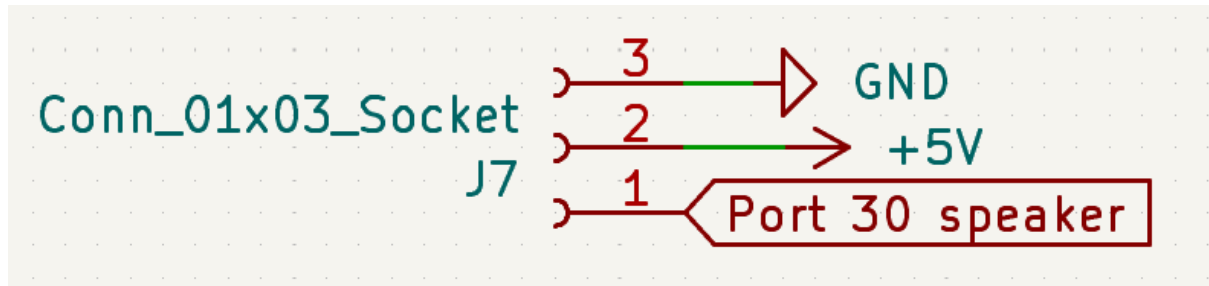
Enfin, l'appui du joueur sur le buzzer est reconnu par un switch de clavier relié à un port du microcontrôleur.



En conséquence, 2 LED seront allumés pour assurer un état visuel. Chacune des LED est branchée en série avec une résistance de protection adéquate.



En plus des LED, un indicateur sensoriel KY-012 est implémenté pour détendre l'atmosphère.



4. Mécanique

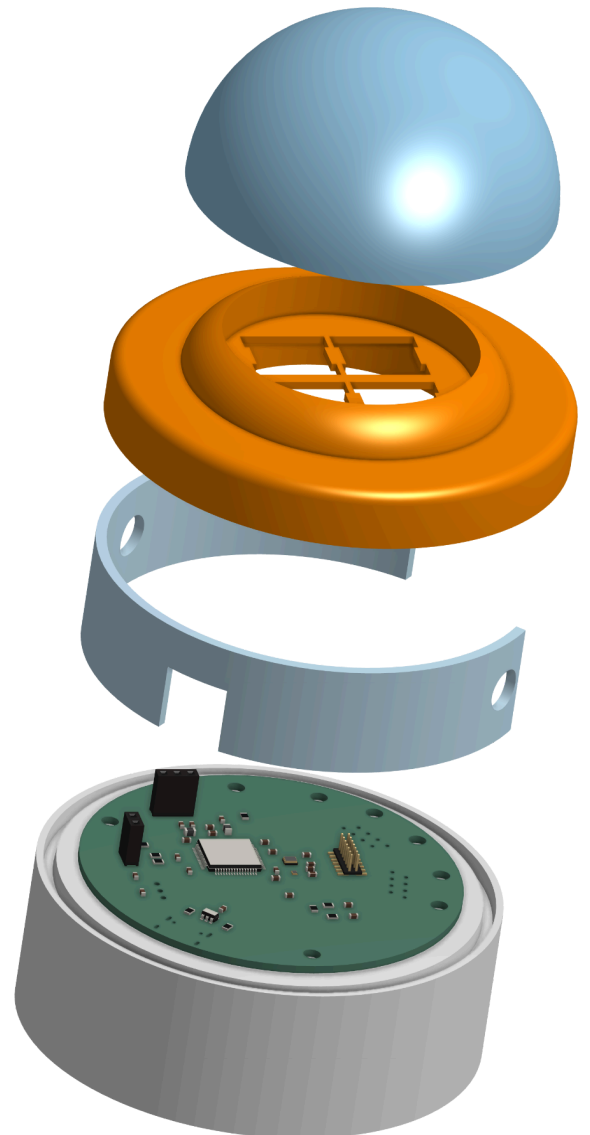
La conception mécanique du buzzer a été entièrement réalisée sur le logiciel de CAO OnShape. L'objectif principal était de créer un boîtier robuste, ergonomique et fonctionnel, capable de loger la carte électronique (conçue sous KiCad) tout en offrant une expérience utilisateur agréable et réactive, typique d'un jeu télévisé ou d'un blindtest.

Le choix s'est porté sur une structure modulaire imprimable en 3D, divisée en plusieurs pièces distinctes pour faciliter le montage, la maintenance et l'intégration des composants.

4.1 Architecture globale du boîtier

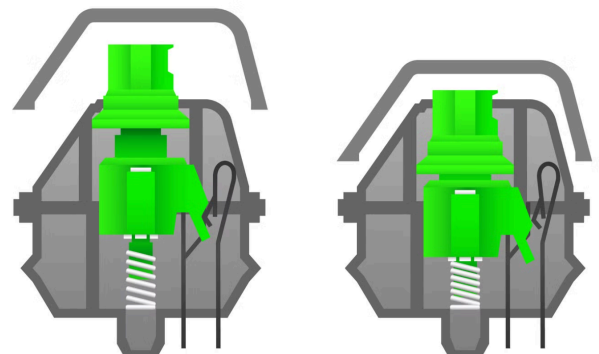
Le boîtier se compose de quatre éléments principaux :

- Le bouton-poussoir : Une large surface d'appui optimisée pour un déclenchement rapide.
- Le socle des switch : Il guide le bouton poussoir et assure la rigidité structurelle en accueillant 4 switches.
- La fenêtre de connectique et lumière : Une plaque amovible dédiée au passage des câbles (2x RJ45 pour le bus CAN et 1x USB-C pour l'alimentation et la communication PC).
- La base inférieure : Elle sert de châssis pour fixer la carte PCB et fermer le boîtier.



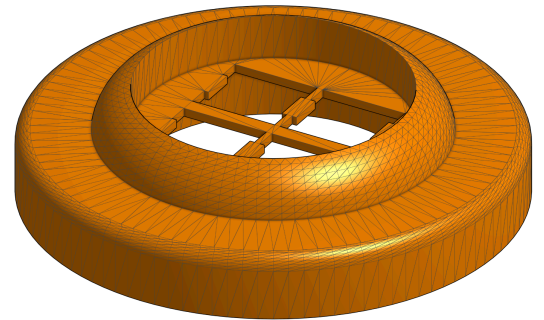
4.2 Choix technologique : Le mécanisme de déclenchement

Pour garantir une excellente réactivité et une grande durabilité, nous avons choisi de ne pas utiliser un simple bouton-poussoir standard, mais un réseau de 4 switches de clavier mécanique disposés en matrice carrée. Lors de la phase d'étude, l'alternative d'un déclenchement par barrière infrarouge (optique) a été écartée au profit des switchs mécaniques pour trois raisons majeures :



- Retour tactile : L'infrarouge n'offre aucune résistance physique. Pour un jeu de rapidité (*blindtest*), le "clic" physique du switch mécanique est indispensable pour confirmer instantanément à l'utilisateur que son appui a été enregistré et ils offrent une course précise.
- Robustesse et simplicité : Contrairement aux capteurs optiques, sensibles à la lumière ambiante et à la poussière, le switch mécanique est une solution passive, fiable et ultra-robuste face aux appuis brusques, en effet ces interrupteurs sont conçus pour supporter des millions de pressions répétées. De plus, l'utilisation de 4 points d'appui répartis sous le large bouton évite que celui-ci ne bascule ou ne se coince s'il est actionné sur un bord plutôt qu'au centre.
- Démarche durable : Ce choix s'inscrit pleinement dans une démarche d'éco-conception. Les quatre switches utilisés ont été récupérés sur un clavier d'ordinateur défectueux destiné au rebut. Cette revalorisation de composants a permis d'annuler le coût de cette fonction tout en limitant l'empreinte carbone de notre prototype.

Le design intègre des empreintes spécifiques dans la structure mécanique pour clipser et maintenir fermement ces 4 switches, assurant un alignement parfait avec les contacts électriques du PCB.

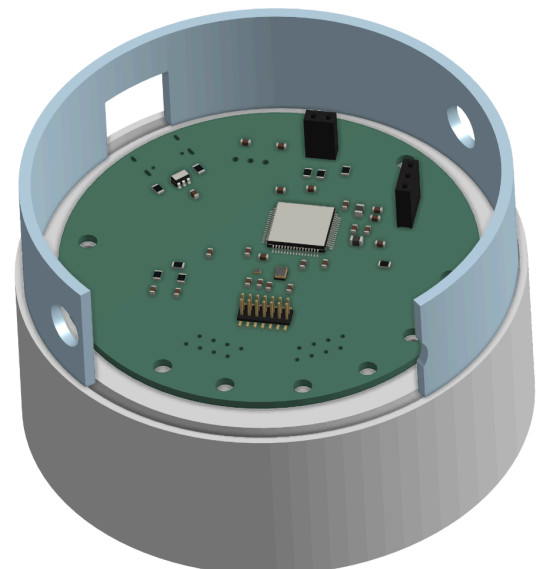


4.3 Intégration Hardware et Ergonomie

La conception a nécessité une communication étroite avec la partie électronique (KiCad) pour s'assurer que les contraintes physiques soient respectées :

- Fixation du PCB : La base inférieure intègre des plots de fixation ajustés aux dimensions de la carte électronique.
- Diffusion lumineuse : Le design intègre des ouvertures pour laisser passer le signal visuel des LED lorsque le buzzer est actif, répondant ainsi directement au cahier des charges.
- Gestion de la connectique : Pour la liaison filaire en bus (RJ45) et le contrôle PC (USB-C), une paroi amovible a été dessinée. Elle permet de maintenir fermement les connecteurs en place et de dissimuler proprement les câbles tout en évitant les tractions directes sur les soudures du PCB.

Passerelle CAO Électronique-Mécanique : Afin de garantir un alignement parfait et d'éviter toute erreur de cotation manuelle, la carte électronique conçue sous KiCad a été exportée au format 3D standardisé **.step**, puis importée directement au sein de notre espace de travail OnShape. Cette intégration numérique nous a



permis de modéliser le boîtier directement autour de la géométrie réelle du PCB.

Grâce à cette méthode, les centres exacts des trous de fixation du circuit imprimé (d'un diamètre de 3,2 mm) ont pu être référencés avec précision dans le contexte d'assemblage pour concevoir les plots de perçage de la base inférieure. De la même manière, le positionnement et les dimensions des ouvertures de la plaque amovible ont été calqués directement sur l'emplacement des connecteurs physiques (ports RJ45 et USB-C), assurant ainsi un ajustement parfait sans jeu structurel parasite ni contrainte mécanique sur les soudures lors du branchement des câbles.

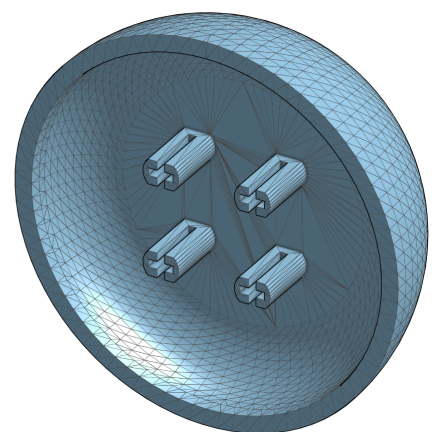
4.4 Assemblage final et perspectives de fabrication

L'assemblage des différentes pièces se fait par emboîtement et vissage, garantissant la maintenance simplifiée du système en cas de besoin (changement d'un switch ou réparation du PCB).

Le matériau utilisé pour l'impression 3d est le PLA, offrant un excellent compromis entre rigidité, facilité d'impression et résistance aux chocs lors des sessions de jeu intensives.

Lors des premières phases de prototypage du bouton principal, nous avons été confrontés à un problème de dimensionnement critique au niveau des tiges du bouton supérieur. Si chaque switch s'insérait parfaitement dans le socle en matrice carrée, l'écartement entre chaque tige du bouton supérieur s'est révélé insuffisant. En pratique, ce manque d'espace faisait qu'il était impossible de faire rentrer deux switches côte à côte simultanément, ces derniers se bloquant mutuellement.

Pour résoudre ce défaut de conception, nous avons modifié les écartements des tiges sur notre logiciel de CAO afin de réajuster l'entraxe et d'offrir l'espace nécessaire à l'insertion fluide des quatre composants.



5. Conclusion

Le projet de développement du système de buzzers interactifs pour jeux a permis d'aboutir à un prototype répondant aux exigences du cahier des charges initial. Nous avons observé la connexion entre trois disciplines : le développement logiciel (software), la conception de circuits électroniques (hardware) et la modélisation mécanique.

Bilan technique et scientifique

- **Software** : L'intégration de la pile USB HID sous STM32CubeIDE a validé la fonctionnalité de contrôle direct du PC (mise en pause automatique de l'ordinateur via la touche Espace). La gestion robuste du protocole FDCAN et de l'algorithme anti-collision (générant des identifiants aléatoires via l'horloge RTC) a garanti une communication asynchrone fiable et réactive sans serveur central.
- **Hardware** : La conception sous KiCad a permis de router une carte électronique, intégrant des transceivers CAN TJA1050, une alimentation filtrée de 5V vers 3,3V ainsi que des protections ESD indispensables contre les décharges électrostatiques dues aux branchements répétés.
- **Mécanique** : En exploitant la passerelle CAO entre KiCad et OnShape (via l'export de fichiers .step), le boîtier modulaire imprimé en PLA s'ajuste à la géométrie réelle de la carte. L'ajustement en cours de prototypage de l'entraxe des tiges du bouton poussoir a résolu les conflits de blocage mutuel des interrupteurs.

Responsabilité environnementale (Up-Cycling)

Ce projet s'est inscrit dans une démarche d'éco-conception. Le choix d'abandonner une barrière infrarouge active au profit d'un réseau passif de 4 switchs mécaniques récupérés sur un clavier d'ordinateur défectueux a permis de donner une seconde vie à des composants destinés au rebut. Cette démarche d'économie circulaire a annulé le coût de cette fonction tout en réduisant l'empreinte carbone du prototype.

Perspectives d'évolutions futures

Bien que le système filaire actuel via connecteurs RJ45 réponde aux critères de robustesse et de résistance aux bruits, plusieurs axes d'amélioration ont été identifiés pour transformer ce prototype en un produit fini plus ergonomique et évolutif:

1. **Scalabilité et connectivité sans fil (BLE)** : Transitionner vers une architecture Bluetooth Low Energy (BLE) afin de rendre le système 100% nomade, éliminer les contraintes de câblage sur le bus et faciliter l'extension à plus de 8 buzzers simultanément.
2. **Interface utilisateur enrichie** : Intégrer un mini-écran OLED sur chaque boîtier afin d'afficher directement le statut de la carte, le score en temps réel des équipes et des signaux audio personnalisés via des profils sonores uniques.
3. **Robustesse matérielle avancée** : Ajouter un module de retour haptique par vibration interne pour notifier physiquement le joueur en cas de verrouillage du système par un adversaire, et concevoir un joint en silicone d'étanchéité (spill-proof) pour protéger l'électronique des projections de liquides lors des sessions de jeu intensives.

ANNEXE

Code STM32CubeIDE

- Inclusions fichiers/bibliothèques:

```
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "usb_device.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include "usbd_hid.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* USER CODE END Includes */
```

- Variables et prototypes:

```

/* Private variables -----*/
FDCAN_HandleTypeDef hfdcan1;

RTC_HandleTypeDef hrtc;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
extern USB_D_HandleTypeDef hUsbDeviceFS;

//Variables FDCAN
FDCAN_FilterTypeDef sFilterConfig;
FDCAN_TxHeaderTypeDef TxHeader;
FDCAN_RxHeaderTypeDef RxHeader;
uint8_t TxData[8];
uint8_t RxData[8];

RTC_TimeTypeDef rtc_time;

//Variables pour le mode multijoueur
uint8_t system_blocked = 0;
uint32_t last_block_time = 0;
char UART_buf[60];
/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_FDCAN1_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_RTC_Init(void);
/* USER CODE BEGIN PFP */
void initFilter(void);
uint32_t generateRandomStdId(void);
void generateStdIdHeader(FDCAN_TxHeaderTypeDef *pHeader);
void initpHeader(FDCAN_TxHeaderTypeDef *pHeader, uint32_t DLC);
void send_spacebar(void);
/* USER CODE END PFP */

```

- Initialisation des filtres et des en-têtes CAN: La fonction 'initFilter' initialise les filtres CAN, tandis que 'initpHeader' et 'generateStdIdHeader' initialisent et génère des en-têtes avec des identifiants standard aléatoires basés sur l'heure RTC.

```
// Initialisation du filtre FDCAN
void initFilter(void) {
    sFilterConfig.IdType = FDCAN_STANDARD_ID;
    sFilterConfig.FilterIndex = 0;
    sFilterConfig.FilterType = FDCAN_FILTER_MASK;
    sFilterConfig.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
    sFilterConfig.FilterID1 = 0x000;
    sFilterConfig.FilterID2 = 0x000;

    HAL_FDCAN_ConfigFilter(&hfdcan1, &sFilterConfig);
}

//Fonction de génération d'ID
uint32_t generateRandomStdId() {
    RTC_DateTypeDef rtc_date;

    HAL_RTC_GetTime(&hrtc, &rtc_time, RTC_FORMAT_BIN);
    HAL_RTC_GetDate(&hrtc, &rtc_date, RTC_FORMAT_BIN);

    // Création d'une seed basée sur le temps
    uint32_t seed = rtc_time.Hours * 3600 + rtc_time.Minutes * 60 + rtc_time.Seconds + rtc_time.SubSeconds;
    srand(seed);

    uint32_t randomBits = rand() & 0xFFFFFFFF;
    uint32_t lowerBits = 0x5;
    uint32_t stdId = (randomBits << 4) | lowerBits;

    return stdId & 0x7FF; //Masque pour rester sur 11 bits (Standard ID)
}
```

```
void generateStdIdHeader(FDCAN_TxHeaderTypeDef *pHeader) {
    pHeader->Identifiant = generateRandomStdId();
}

void initpHeader(FDCAN_TxHeaderTypeDef *pHeader, uint32_t DLC) {
    pHeader->Identifiant = generateRandomStdId();
    pHeader->IdType = FDCAN_STANDARD_ID;
    pHeader->TxFrameType = FDCAN_DATA_FRAME;
    pHeader->DataLength = FDCAN_DLC_BYTES_2;
    pHeader->ErrorStateIndicator = FDCAN_ESI_ACTIVE;
    pHeader->BitRateSwitch = FDCAN_BRS_OFF;
    pHeader->FDFormat = FDCAN_CLASSIC_CAN;
    pHeader->TxEventFifoControl = FDCAN_NO_TX_EVENTS;
    pHeader->MessageMarker = 0;
}
```

- Fonctions d'appui espace et de callback:

```
//Fonction pour mettre la vidéo/son en pause via USB (clavier HID)
void send_spacebar(void) {
    uint8_t hid_report[4] = {0};

    //Appui sur la touche Espace (code HID:0x2C)
    hid_report[2] = 0x2C;
    USB_HID_SendReport(&hUsbDeviceFS, hid_report, sizeof(hid_report));

    HAL_Delay(30); //Petit délai pour simuler un vrai doigt

    //Relâcher la touche
    hid_report[2] = 0x00;
    USB_HID_SendReport(&hUsbDeviceFS, hid_report, sizeof(hid_report));
}
```

```

//Un adversaire a buzzé: callback lors de la réception d'un message FDCAN
void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
{
    if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_NEW_MESSAGE) != RESET)
    {
        if (HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK)
        {
            //Quelqu'un a buzzé
            system_blocked = 1;
            last_block_time = HAL_GetTick();

            //On met en pause l'ordinateur
            send_spacebar();

            sprintf(UART_buf, ">> CAN RECU (ID: %lx) ! JE SUIS BLOQUE \r\n", RxHeader.Identifiant);
            HAL_UART_Transmit(&huart2, (unsigned char*)UART_buf, strlen(UART_buf), 100);
        }
    }
}
/* USER CODE END 0 */

```

- Fonction principale:

```

int main(void)
{
    /* USER CODE BEGIN 1 */
    uint8_t prev_bouton_etat = GPIO_PIN_SET;
    uint8_t active_bouton = GPIO_PIN_SET;
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_FDCAN1_Init();
    MX_USART2_UART_Init();
    MX_USB_Device_Init();
    MX_RTC_Init();
    /* USER CODE BEGIN 2 */
    //Extinction forcée au démarrage pour les LED et buzzer
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10 | GPIO_PIN_11, GPIO_PIN_RESET);

    initFilter();
    initpHeader(&TxHeader, 2);
}

```

```

// Lancement du FDCAN
HAL_FDCAN_Start(&hfdcan1);
HAL_FDCAN_ActivateNotification(&hfdcan1, FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);

HAL_UART_Transmit(&huart2, (unsigned char*)UART_buf, strlen(UART_buf), HAL_MAX_DELAY);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    //Déblocage après 3 secondes écoulées
    if (system_blocked && (HAL_GetTick() - last_block_time > 3000)) {
        system_blocked = 0;
    }

    //on écoute le bouton si on n'est pas bloqué (PA10)
    if (!system_blocked)
    {
        active_bouton = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);

        //Le joueur a buzzé en premier
        if(active_bouton == GPIO_PIN_RESET && prev_bouton_etat == GPIO_PIN_SET)
        {
            //Mise en pause de la vidéo
            send_spacebar();

            //Blocage des autres sur le CAN
            HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &TxHeader, TxData);

```

```

        //Allumage LED et speaker
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);

        HAL_UART_Transmit(&huart2, (unsigned char*)UART_buf, strlen(UART_buf), HAL_MAX_DELAY);

        HAL_Delay(1000);
        //Extinction du speaker
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);

        HAL_Delay(2000);

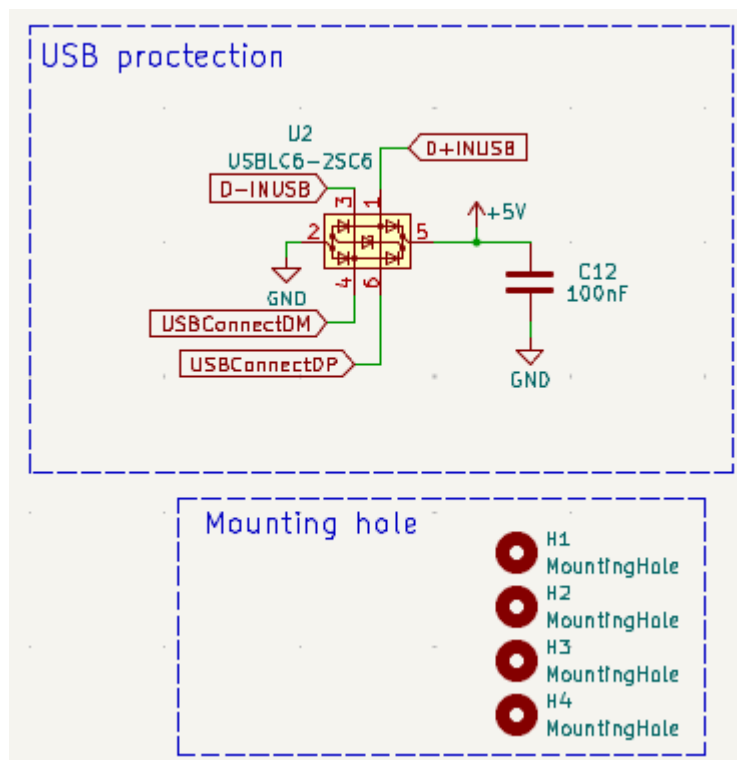
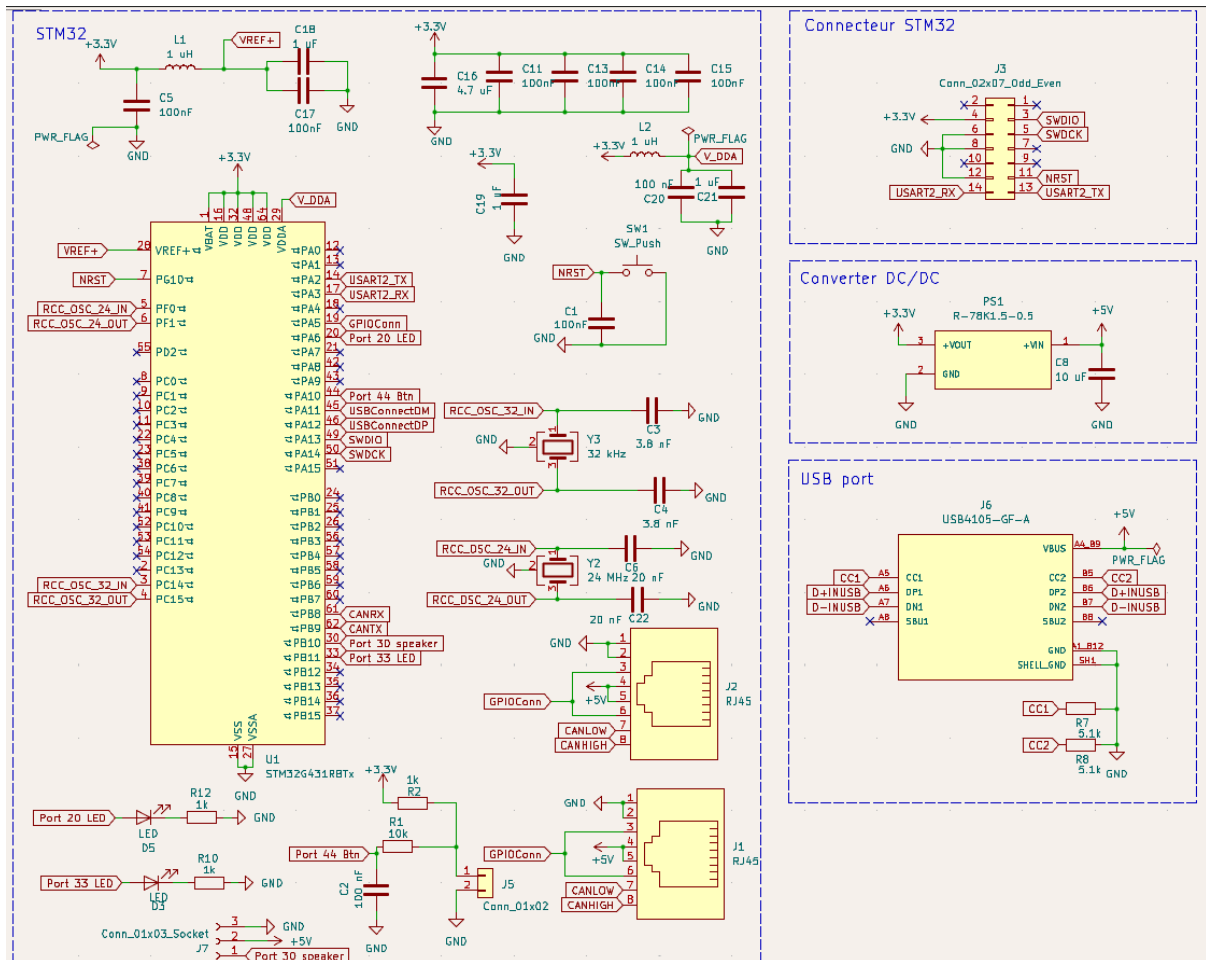
        //Extinction des LED
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, GPIO_PIN_RESET);

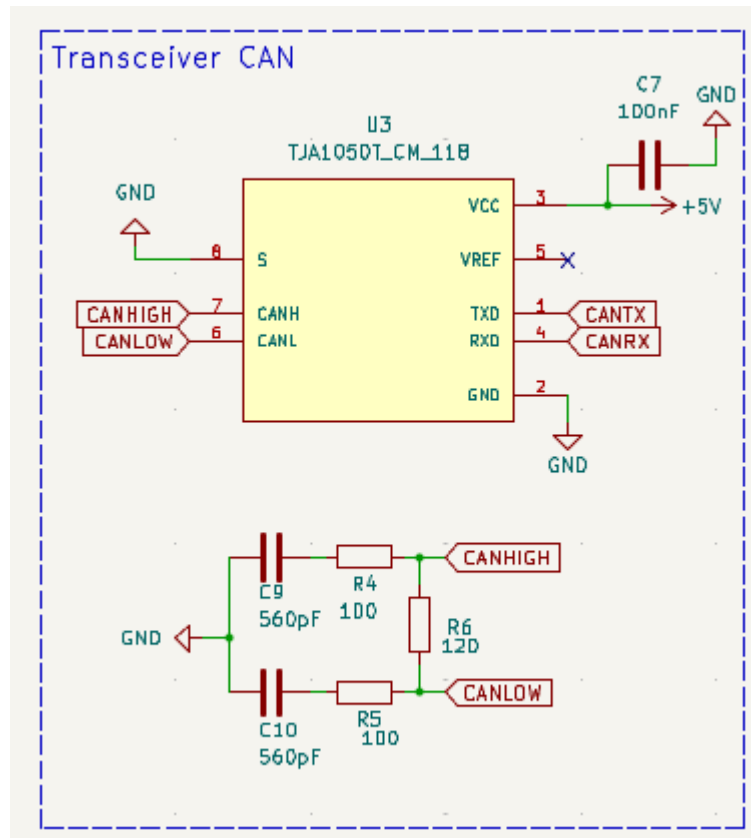
        //Préparation d'un nouvel ID
        generateStdIdHeader(&TxHeader);
        system_blocked = 0;
    }
    prev_bouton_etat = active_bouton;
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

```

Schematic Editor de KiCad.





PCB Editor

